Intel 8086 Microprocessor

Architecture, Memory Organisation & System Design

A Comprehensive Study Guide

Based on Chapter 2 & Chapter 3 — Intel 8086 Microprocessor Textbook

Covers: Memory Organisation · Segmentation · Pins & Signals · System Design · Even/Odd Banks

---

Chapter 3: Introduction to the Intel 8086

3.1 Memory Organisation and Word Addressing

The memory of an 8086-based microcomputer is organised as bytes. Each byte can be uniquely addressed with 20-bit addresses ranging from 00000_{16} through $FFFFF_{16}$. An 8086 16-bit word consists of any two consecutive bytes; the low-addressed byte is the low byte of the word and the high-addressed byte contains the high byte.

Low byte of the word	High byte of the word
07_{16}	26_{16}
Address 00520_{16}	Address 00521_{16}

Figure: Word stored at even address.

> *Low byte of the word (07_{16}) is at Address 00520_{16}; high byte (26_{16}) is at Address 00521_{16}. The 16-bit word stored at even address 00520_{16} is therefore 2607_{16}.*

Next, consider a word stored at an odd address:

Low byte of the word	High byte of the word
05_{16}	$3F_{16}$
Address 01257_{16}	Address 01258_{16}

Figure: Word stored at odd address.

> *Low byte (05_{16}) is at Address 01257_{16}; high byte ($3F_{16}$) is at Address 01258_{16}. The 16-bit word stored at odd address 01257_{16} is $3F05_{16}$.*

Note that for word addresses the programmer uses the low-order address (odd or even) to specify the whole 16-bit word.

The 8086 always accesses a 16-bit word to or from memory. The 8086 can read a 16-bit word in one operation if the first byte of the word is at an even address. On the other hand, the 8086 must perform two memory accesses to two consecutive memory even addresses, if the first byte of the word is at an odd address. In this case the 8086 discards the unwanted bytes of each.

For example, consider MOV BX, ADDR. Note that the X or H (or L) following the 8086 register name in an instruction indicates whether the transfer is 16-bit or 8-bit. For example, MOV ADDR, BX moves the contents of the 20-bit physical memory location addressed by ADDR into the 8086 16-bit register BX. Now, if ADDR is a 20-bit even address such as 30024_{16}, then this MOV instruction loads the low (BL) and high (BH) bytes of the 8086 16-bit register BX with the contents of memory locations 30024_{16} and 30025_{16} respectively, in a single access.

Now, if ADDR is an odd address such as 40005_{16}, then the MOV BX, ADDR instruction loads BL and BH with the contents of memory locations 40005_{16} and 40006_{16} respectively, in two accesses. Note that the 8086 accesses locations 40004_{16} and 40005_{16} in the first operation but discards the contents of 40004_{16}, and in the second operation accesses 40006_{16} and 40007_{16} but ignores the contents of 40007_{16}.

3.1 The 8086 and 8088 Microprocessor

The 8086 family consists of two types of 16-bit microprocessors — the 8086 and 8088. The main difference is how the processors communicate with the outside world. The 8088 has an 8-bit external data path to memory and I/O, while the 8086 has a 16-bit external data path. This means that the 8088 will have to do two read operations to read a 16-bit word into memory. In most other respects, the processors are identical. Note that the 8088 accesses memory in bytes. No alterations are needed to run software written for one microprocessor on the other. Because of similarities, only the 8086 will be considered here. The 8088 is used in designing the IBM Personal Computer.

An 8086 can be configured as a small uniprocessor system (minimum mode if the MN/$\overline{\text{MX}}$ pin is tied to HIGH) or as a multiprocessor system (maximum mode when MN/$\overline{\text{MX}}$ pin is tied to LOW). In a given system, the MN/$\overline{\text{MX}}$ pin is permanently tied to

either HIGH or LOW. Some of the 8086 pins have dual functions depending on the selection of the MN/MX pin level.

In the minimum mode (MN/MX pin high), these pins transfer control signals directly to memory and input/output devices. In the maximum mode (MN/MX pin low), these same pins have different functions which facilitate multiprocessor systems. In the maximum mode, the control functions normally present in minimum mode are assumed by a support chip, the 8288 bus controller.

3.2 8086 Architecture — Segmentation

The 8086 uses a segmented memory model. The physical 20-bit address is computed from a 16-bit segment register and a 16-bit offset. This allows access to the full 1 MB address space using 16-bit internal registers.

The stack segment address is calculated from SS and SP for stack instructions such as PUSH and POP. The programmer can use the BP register instead of SP for accessing the stack using the based addressing mode. In this case the 20-bit physical stack address is calculated from BP and SS.

The DS register points to the current data segment; operands for most instructions are fetched from this segment. The 16-bit contents of Source Index (SI) or Destination Index (DI) are used as offset for computing the 20-bit physical address.

The ES register points to the extra segment in which data (in excess of 64K pointed to by DS) is stored. String instructions always use ES and DI to determine the 20-bit physical address for the destination.

The segments can be continuous, partially overlapped, fully overlapped, or disjoint. An example of how five segments (segment 0 through segment 4) may be stored in physical memory is shown below:

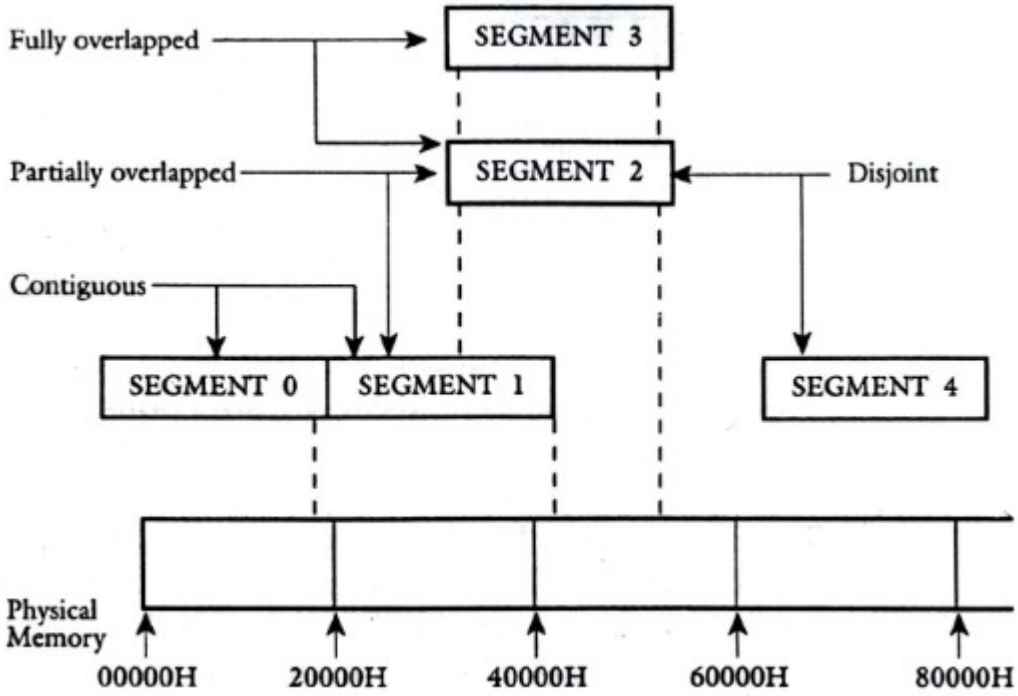


Figure 3 — Segment Relationships in Physical Memory: A diagram showing physical memory from 00000H to 80000H.

> *SEGMENT 0 and SEGMENT 1 are contiguous (adjacent), starting at 00000H and 20000H respectively. SEGMENT 2 starts at 20000H and is partially overlapped with SEGMENT 1. SEGMENT 3 starts at 20000H and is fully overlapped with SEGMENT 2. SEGMENT 4 starts at 60000H and is disjoint from SEGMENT 2. Labels indicate: Fully overlapped → SEGMENT 3; Partially overlapped → SEGMENT 2; Contiguous → SEGMENT 0 and 1; Disjoint → SEGMENT 4.*

Typical examples of values of segments should then be selected based on physical addresses starting at 00000_{16}, 00010_{16}, 00020_{16}, 00030_{16}, ... $FFFF0_{16}$. A physical memory location may be mapped into (contained in) one or more logical segments. Many applications can be written to simply initialise the segment registers and then forget them.

Chapter 3 · System Design Using the 8086

This section covers the basic concepts associated with interfacing the 8086 to its support chips such as memory and I/O. Topics such as timing diagrams and 8086 pins and signals will also be included.

3.8.1 Pins and Signals

The 8086 pins and signals are shown in Figure 3.6. Unless otherwise indicated, all 8086 pins are TTL compatible. As mentioned before, the 8086 can operate in two modes. These are minimum mode (uniprocessor system — single 8086) and maximum mode (multiprocessor system — more than one 8086). MN/M$\bar{X}$ is an input pin used to select one of these modes.

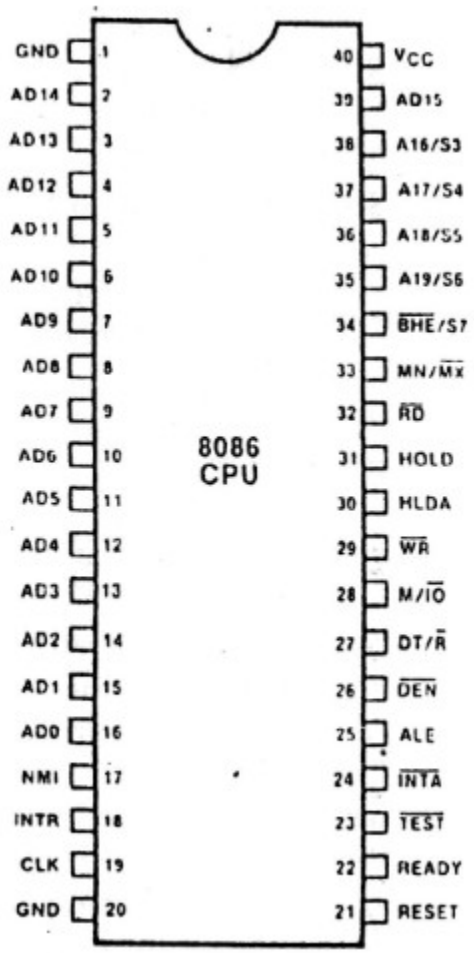
When MN/M$\bar{X}$ is HIGH, the 8086 operates in the minimum mode. In this mode, the 8086 is configured (that is, pins are defined) to support small, single processor systems using a few devices that use the system bus.

When MN/M$\bar{X}$ is LOW, the 8086 is configured (that is, pins are defined in the maximum mode) to support multiprocessor systems. In this case, the Intel 8288 bus controller is added to the 8086 to provide bus controls and compatibility with the multibus architecture. Note that in a particular application, the MN/M$\bar{X}$ must be tied to either HIGH or LOW.

AD0–AD15 lines are a 16-bit multiplexed address/data bus. During the first clock cycle AD0–AD15 are the low order 16 bits of address. The 8086 has a total of 20 address lines. The upper four lines are multiplexed with the status signals for the 8086. These are the A16/S3, A17/S4, A18/S5, and A19/S6. During the first clock period of a bus cycle (read or write cycle), the entire 20-bit address is available on these lines. During all other clock cycles for memory and I/O operations, AD15–AD0 contain the 16-bit data, and S3, S4, S5, and S6 become status lines.

S3 and S4 lines are decoded as follows:

A17/S4	A16/S3	Function
0	0	Extra segment
0	1	Stack segment
1	0	Code or no segment
1	1	Data segment



The common signals, minimum mode signals, and maximum mode signals are summarised in the table below:

Name	Function	Type
AD15–AD0	Address/Data Bus	Bidirectional, 3-State
A19/S6–A16/S3	Address/Status	Output, 3-State
$\overline{BHE}$/S7	Bus High Enable / Status	Output, 3-State
MN/$\overline{MX}$	Minimum/Maximum Mode Control	Input
$\overline{RD}$	Read Control	Output, 3-State
$\overline{TEST}$	Wait On Test Control	Input
READY	Wait State Control	Input
RESET	System Reset	Input
NMI	Non-Maskable Interrupt Request	Input
INTR	Interrupt Request	Input
CLK	System Clock	Input
VCC	+5V	Input
GND	Ground	—

Minimum Mode Signals (MN/$\overline{MX}$= VCC):

Name	Function	Type
HOLD	Hold Request	Input
HLDA	Hold Acknowledge	Output
$\overline{WR}$	Write Control	Output, 3-State
M/$\overline{IO}$	Memory/IO Control	Output, 3-State
DT/$\overline{R}$	Data Transmit/Receive	Output, 3-State
$\overline{DEN}$	Data Enable	Output, 3-State

Name	Function	Type
ALE	Address Latch Enable	Output
INTA	Interrupt Acknowledge	Output

Maximum Mode Signals (MN/MX= GND):

Name	Function	Type
RQ/GT1,0,	Request/Grant Bus Access Control	Bidirectional
LOCK	Bus Priority Lock Control	Output, 3-State
S2-S0	Bus Cycle Status	Output, 3-State
QS1, QS0	Instruction Queue Status	Output

3.8.2 Maximum Mode — Status and Bus Control

In the maximum mode, some of the 8086 pins in the minimum mode are redefined. For example, pins HOLD, HLDA, WR, M/IO, DT/R, DEN, ALE, and INTA in the minimum mode are redefined as RQ/GT0, RQ/GT1, LOCK, S2, S1, S0, QS0, and QS1, respectively. S2, S1, S0 are 8086 outputs and are decoded as follows:

$\bar{S}2$	$\bar{S}1$	$\bar{S}0$	Function
0	0	0	Interrupt Acknowledge
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	Halt
1	0	0	Code access
1	0	1	Read memory
1	1	0	Write memory
1	1	1	Inactive

RQ/GT0, RQ/GT1: These request/grant pins are used by other local bus masters to force the processor to release the local bus at the end of the processor's current bus cycle. Each pin is bidirectional, with RQ/GT0 having higher priority than RQ/GT1. These pins have internal pull-up resistors so that they may be left unconnected. The request/grant function of the 8086 works as follows:

1. A pulse (one clock wide) from another local bus master (RQ/GT0 or RQ/GT1 pins) indicates a local bus request to the 8086.
2. At the end of 8086 current bus cycle, a pulse (one clock wide) from the 8086 to the requesting master indicates that the 8086 has relinquished the system bus and tristated the outputs. Then the new bus master subsequently relinquishes control of the system bus by sending a LOW on RQ/GT0 or RQ/GT1 pins. The 8086 then regains bus control.

LOCK: The 8086 outputs LOW on the LOCK̄ pin to prevent other bus masters from gaining control of the system bus.

3.8.2b 8086 Address and Data Bus Concepts

The majority of memory and I/O chips capable of interfacing to the 8086 require a stable address for the duration of the bus cycle. Therefore, the address on the 8086 multiplexed address/data bus during T1 should be latched. The latched address is then used to select the desired I/O or memory location. Note that the 8086 has a 16-bit multiplexed address and data bus, while the 8085's 8-bit data lines and LOW address byte are multiplexed. Hence, the multiplexed bus components of the 8085 family are not applicable to the 8086.

To demultiplex the bus, the 8086 provides an ALE (Address Latch Enable) signal to capture the address in either the 8282 (noninverting) or 8283 (inverting) 8-bit bistable latches. These latches propagate the address through to the outputs while ALE is HIGH and latch the address in the following edge of ALE. This only delays address access and chip select decoding by the propagation delay of the latch.

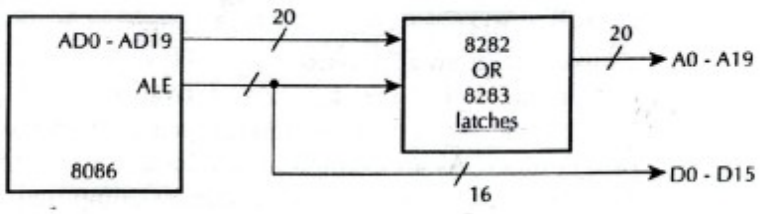


FIGURE 3.10 Separate address and data buses.

FIGURE 3.10 — Separate Address and Data Buses.

> *Block diagram showing the 8086 connected to 8282 or 8283 latches via a 20-bit AD0–AD19 bus and ALE signal. The latches output A0–A19 (20-bit address) separately and D0–D15 (16-bit data bus) directly from the 8086.*

The programmer views the 8086 memory address space as a sequence of one million bytes in which any byte may contain an eight-bit data element and any two consecutive bytes may contain a 16-bit data element. There is no constraint on byte or word addresses (boundaries). The address space is physically implemented on a 16-bit data bus by dividing the address space into two banks of up to 512K bytes as shown in Figure 3.11. These banks can be selected by BHE and A0 as follows:

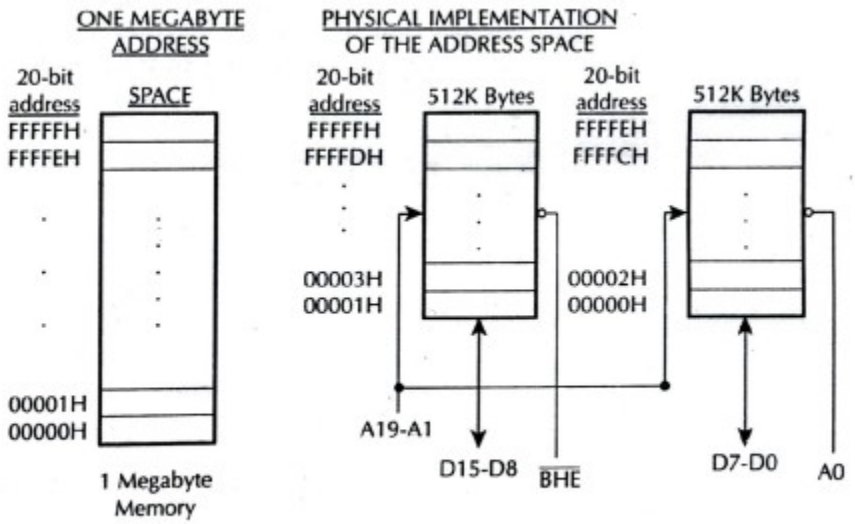


FIGURE 3.11 8086 memory.

FIGURE 3.11 — 8086 Memory Organisation.

> *Left panel shows 1 Megabyte address space (20-bit, FFFFFH down to 00000H). Right panel shows the physical implementation: Two 512K-byte banks. The odd bank (D15–D8, selected by BHE) holds odd addresses (00001H, 00003H, … FFFFBH, FFFFFDH, FFFFFH). The even bank (D7–D0, selected by A0) holds even addresses (00000H, 00002H, … FFFFAH, FFFFCH, FFFFDEH). A19–A1 address individual bytes within each bank.*

BHE	A0	Byte Transferred
0	0	Both bytes
0	1	Upper byte to/from odd address
1	0	Lower byte to/from even address
1	1	None

One bank is connected to D7–D0 and contains all even addressed bytes (A0 = 0). The other bank is connected to D15–D8 and contains odd-addressed bytes (A0 = 1). A particular byte in each bank is addressed by A19–A1.

The even-addressed bank is enabled by LOW A0 and data bytes transferred over D7–D0 lines. The 8086 outputs HIGH on BHE (Bus High Enable) and thus disables the odd-addressed bank. The 8086 outputs LOW on BHE to select the odd-addressed bank and HIGH on A0 to disable the even-addressed bank. This directs the data transfer to the appropriate half of the data bus. Activation of A0 and BHE is performed by the 8086 depending on odd or even addresses and is transparent to the programmer.

As an example, consider execution of the instruction MOV DH, [BX]. Suppose the 20-bit address computed by BX and DS is even. The 8086 outputs LOW on A0 and HIGH on BHE. This will select the even-addressed bank. The content of the selected memory is placed on the D7–D0 lines by the memory chip. The 8086 reads this data via D7–D0 and automatically places it in DH.

Next, consider accessing a 16-bit word by the 8086 with low byte at an even address as shown in Figure 3.12.

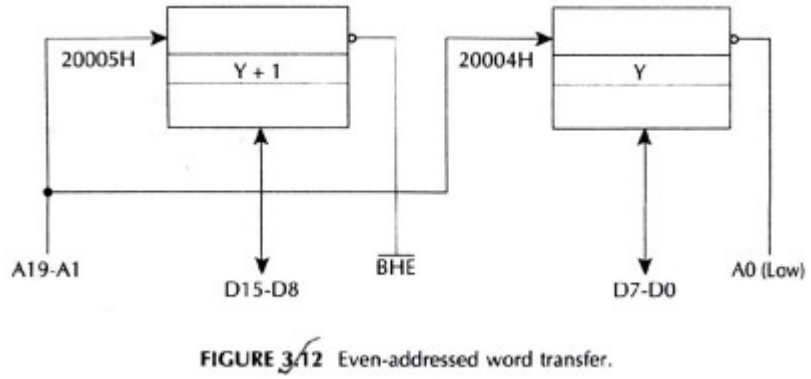


FIGURE 3.12 — Even-Addressed Word Transfer.

> *Diagram showing a single bus cycle. The odd memory bank (A19–A1, D15–D8, BHE) holds value Y+1 at address 20005H. The even memory bank (D7–D0, A0 Low) holds value Y at address 20004H. Both banks are accessed simultaneously when A0 = LOW and BHE = LOW.*

For example, suppose that the 8086 executes the instruction MOV [BX], CX. Assume [BX] = 0004H, [DS] = 2000H. The 20-bit physical address for the word is 20004H. The 8086 outputs LOW on both A0 and BHE, enabling both banks simultaneously. The 8086 outputs [CL] to D7–D0 lines and [CH] to D15–D8 lines with WR LOW and M/IO HIGH. The enabled memory banks obtain the 16-bit data and write [CL] to location 20004H and [CH] to location 20005H.

Next, consider accessing an odd-addressed 16-bit word by the 8086. For example, suppose the 20-bit physical address computed by the 8086 is 20005H. The 8086 accomplishes this transfer in two bus cycles. In the first bus cycle, the 8086 outputs HIGH on A0, LOW on BHE, and thus enables the odd-addressed bank and disables the even-addressed bank. The 8086 also outputs LOW on RD and HIGH on M/IO pins. In this bus cycle, the odd memory bank places [20005H] on D15–D8 lines. The 8086 reads this data into CL. In the second bus cycle, the 8086 outputs LOW on A0, HIGH on BHE, and thus enables the even-addressed bank and disables the odd-addressed bank.

The 8086 also outputs LOW on RD and HIGH on M/IO pins. The selected even-addressed memory bank places [20006H] on D7–D0 lines. The 8086 reads this data into CH. This is shown in Figure 3.13.

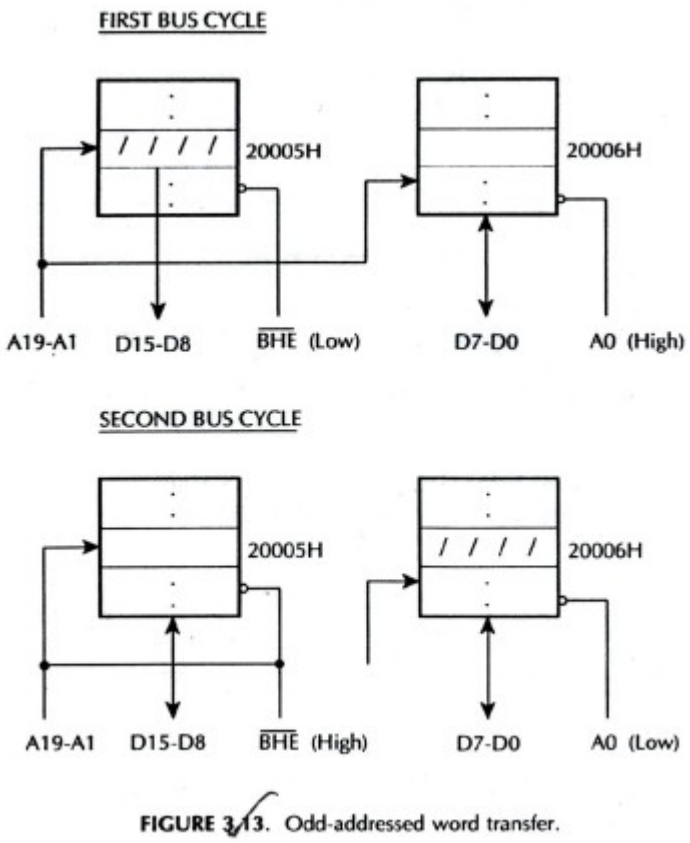


FIGURE 3.13 — Odd-Addressed Word Transfer (Two Bus Cycles.

> *FIRST BUS CYCLE: Odd bank (A19–A1, D15–D8, BHE Low) provides data from address 20005H; even bank (D7–D0, A0 High) is disabled. SECOND BUS CYCLE: Even bank (D7–D0, A0 Low) provides data from address 20006H; odd bank (D15–D8, BHE High) is disabled.*

During a byte read, the 8086 floats the entire D15–D0 lines during portions of T2 cycle even though data are expected on the upper or lower half of the data bus. As will be shown later, this action simplifies the chip select decoding requirements for ROMs and EPROMs. During a byte write, the 8086 will drive the entire 16-bit data bus. The

information on the half of the data bus not transferring data is indeterminate. These concepts also apply to I/O transfers.

If memory or I/O devices are directly connected to the multiplexed bus, the designer must guarantee that the devices do not corrupt the address on the bus during T1. To avoid this, the memory or I/O devices should have an output enable controlled by the 8086 read signal.

Chapter 2 · Even and Odd Memory Banks

2.5 Even and Odd Memory Banks — Detailed Analysis

The 8086 microprocessor uses a 20-bit address to access memory. With a 20-bit address the processor can generate 2^{20} = 1 Mega address. The basic memory word size of the memories used in the 8086 system is 8-bit or 1-byte (i.e., in one memory location an 8-bit binary information can be stored). Hence, the physical memory space of the 8086 is 1Mb (1 Mega-byte).

For the programmer, the 8086 memory address space is a sequence of one mega-byte in which one location stores an 8-bit binary code/data and two consecutive locations store 16-bit binary code/data. But physically (i.e., in the hardware), the 1Mb memory space is divided into two banks of 512kb (512kb + 512kb = 1Mb). The two memory banks are called Even (or Lower) bank and Odd (or Upper) bank.

The 8086-based system will have two sets of memory ICs. One set for even bank and another set for odd bank. The data lines D_0–D_7 are connected to even bank and the data lines D_8–D_{15} are connected to odd bank. The even memory bank is selected by the address line A_0 and the odd memory bank is selected by the control signal <u>BHE</u>. The memory banks are selected when these signals are low (active low). Any memory location in the memory bank is selected by the address line A_1 to A_{19}.

The organisation of memory into two banks and providing bank select signals allows the programmer to read/write the byte (8-bit) operand in any memory address through the 16-bit data bus. It also allows the programmer to read/write the word (16-bit) operand starting from even address or odd address.

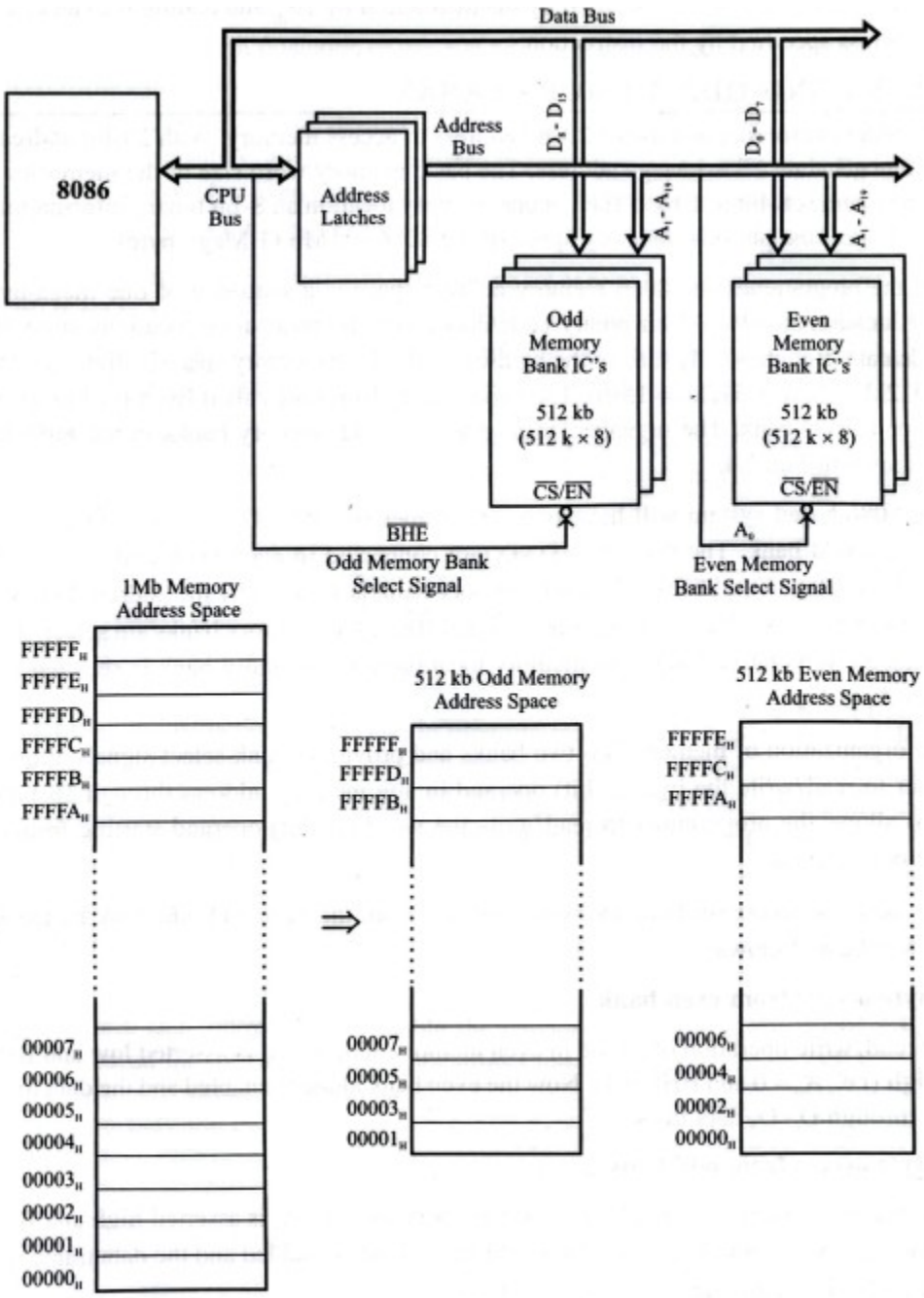


Fig. 2.4 : Organization of even and odd memory banks in 8086-based system.

📊 *FIGURE 2.4 — Organisation of Even and Odd Memory Banks in an 8086-Based System.*

Top block diagram shows the 8086 CPU with its CPU Bus connected to Address Latches. The Address Bus (A_1–A_{19}) feeds both Odd Memory Bank ICs (512k × 8, D_8–D_{15}) and Even Memory Bank ICs (512k × 8, D_0–D_7). The Odd bank is selected by BHE (CS/EN active low) and the Even bank is selected by A_0 (CS/EN active low). Bottom section shows address space mapping: The

1Mb Memory Address Space (00000H–FFFFFH) is split into: 512kb Odd Memory Address Space with odd addresses (00001H, 00003H, 00005H, 00007H … FFFFFH, FFFFDH, FFFFFBH) and 512kb Even Memory Address Space with even addresses (00000H, 00002H, 00004H, 00006H … FFFFAH, FFFFCH, FFFFDEH, FFFFEH).

The memory access for byte and word operand from the even and odd bank by 8086 processor will be as follows:

Case i — Byte Access from Even Bank

For read/write operation of a byte in even memory address, A_0 is asserted low and $\overline{BHE}$ is asserted high (i.e., $A_0 = 0$ and $\overline{BHE} = 1$). Now the even bank alone is enabled and the data transfer takes place through D_0–D_7 data lines.

Case ii — Byte Access from Odd Bank

For read/write operation of a byte in odd memory address, A_0 is asserted high and $\overline{BHE}$ is asserted low (i.e., $A_0 = 1$ and $\overline{BHE} = 0$). Now the odd bank alone is enabled and the data transfer takes place through D_8–D_{15} data lines.

Case iii — Word Access from Even Boundary

For read/write operation of a word (16-bit) in even boundary (i.e., low byte in even address and high byte in next address (odd address)), both A_0 and $\overline{BHE}$ are asserted low (i.e., $A_0 = 0$ and BHE = 0). Now both the memory banks are enabled simultaneously and the processor read/writes the 16-bit operand in one bus cycle through D_0–D_{15} data lines.

Case iv — Word Access from Odd Boundary

For read/write operation of a word (16-bit) in odd boundary (i.e., low byte in odd address and high byte in next address (even address)), the processor executes two bus cycles to read/write the word (16-bit) operand. In the first bus cycle A_0 is asserted high and $\overline{BHE}$ is asserted low (i.e., $A_0 = 1$ and BHE = 0). Now the odd bank alone is enabled and the low byte of the 16-bit operand is read/write through D_8–D_{15} data lines. In the second bus cycle A_0 is asserted low and $\overline{BHE}$ is asserted high (i.e., $A_0 = 0$ and $\overline{BHE} = 1$). Now the even bank alone is enabled and the high byte of the 16-bit operand is read/write through D_0–D_7 data lines.

TABLE 2.8 — Status of A_0 and BHE During Memory Access:

Memory Bank	Operand Type	Status of		Data Lines for Memory Access	No. of Bus Cycles
		A_0	BHE		
Even	Byte	0	1	D_0 – D_7	One
Odd	Byte	1	0	D_8 – D_{15}	One
Even	Word	0	0	D_0 – D_{15}	One
Odd	Word (1st cycle)	1	0	D_8 – D_{15}	First cycle
Odd	Word (2nd cycle)	0	1	D_0 – D_7	Second cycle

> *Note 1: The processor may access the low byte operand via the upper data lines and the high byte operand via the lower data lines, but while placing the operand in the registers it places them in the appropriate locations.*
>
> *Note 2: When word operand is accessed from odd boundary the instruction execution will take extra time due to two bus cycle memory access.*

— End of Study Guide —

Intel 8086 Microprocessor: Architecture, Memory Organisation & System Design